

Kanbien Architecture Discussion

Expanded transcript-style reconstruction of the architecture discussion prior to the learning prompts.

Table of Contents

1. Original Context
2. Front-End Architecture
3. Design System Architecture
4. Platform Layer
5. Core Packages
6. Applications
7. Harness
8. Build and Code Generation
9. Persistence
10. Infrastructure and Deployment
11. Entity Builder Flow
12. Architectural Recommendations

1. Original Context

The goal was to redesign Kanbien after an earlier implementation became bloated. The original stack used HTML/CSS/JavaScript front end seams, Next.js, Node + TypeScript, PostgreSQL, mounted features, an attempted design system, and incomplete deployment automation. The desired future architecture included: • Portable React front end • Strictly governed design system • Modular platform layer • Domain-oriented backend • Entity-driven code generation • AWS + Terraform deployment • Deterministic CRUD generation from entity definitions

2. Front-End Architecture

A React front end should be treated as a reusable product surface rather than application-specific code. The design system should be separated from applications and consumed through published packages. Applications should not directly modify primitives, patterns, or tokens.

3. Design System Architecture

Recommended structure: packages/design-system/ tokens/ foundations/ builders/ resolved/ primitives/ patterns/ components/ resolvers/ Flow: Design Label → Foundation Tokens → Builder Tokens → Primitives → Patterns → Components This allows multiple visual systems to share identical behaviour while presenting different visual identities.

4. Platform Layer

Recommended structure: platform/ server/ http.ts cors.ts rate-limit.ts auth.ts routes.ts mount.ts

Responsibilities: • HTTP startup • Route registration • Middleware • Authentication hooks • Health checks • Logging • Rate limiting • Request context Business logic should not live here.

5. Core Packages

Reusable backend capabilities: packages/core/ jobs/ files/ auth/ audit/ notifications/ validation/ persistence/ Core packages are reusable services shared across applications.

6. Applications

Applications own business behaviour. apps/crm/ entities/ features/ migrations/ app.mount.ts
Applications may run independently while sharing platform and core packages.

7. Harness

The harness acts as the governance and generation layer. Responsibilities: • Capture entity definitions • Validate contracts • Apply governance • Drive code generation • Enforce ownership boundaries

8. Build and Code Generation

Recommended structure: tools/build/ generate-entity.ts generate-crud.ts generate-tests.ts
generate-migration.ts Generation should be deterministic and idempotent. Running generation
repeatedly should not create duplication.

9. Persistence

Persistence should be application-owned. apps/crm/migrations/ Entity definitions describe intent. Generated migrations become part of an ordered migration stream. Suggested approach: • PostgreSQL • SQL-first migrations • Drizzle or similar tooling

10. Infrastructure and Deployment

Recommended structure: infra/ terraform/ aws/ network/ app/ database/ secrets/ ci-cd/ Terraform owns infrastructure provisioning. Application manifests drive deployment behaviour.

11. Entity Builder Flow

Desired workflow: Chat → Harness captures entity definition → Validation → Backend schema generation → API contract generation → Front-end configuration generation → Migration generation → Test generation → App mounting → Platform mounting → CI deployment → Staging → Production The entity becomes the source of truth.

12. Architectural Recommendations

Key recommendation: Do not allow features to become the primary architectural boundary. Instead: Entities define intent. Generators create implementation. Applications own business composition. Platform owns runtime concerns. Infrastructure owns deployment concerns. This creates a cleaner separation of responsibilities and prevents the repo from becoming bloated as it grows.